

BUGUARD



Lean Technologies Penetration Testing (KSA)

Final Report

Lean Technologies

Confidential

Dec Nov 20, 2024



REPORT PROPERTY			
Report Name	Lean Technologies (KSA) Penetration Testing Final Report		
Description	This report describes the security findings that was discover during the Lean Technologies (KSA) Penetration Testing engagement. The PT was conducted on both sandbox and production environment.		
Classification	CONFIDENTIAL		
Issue Date	Nov 20, 2024		
Issue Number	1.2		
Authors	Ziad Elsayed	Fares Walid	Mohamed Ayman
Approved By	Moaaz Hamdi		
Contacts	Youssef@buguard.io	m.hamdi@buguard.io	
Client’s Contacts	yash.pandya@leantech.me		

DOCUMENT CONTROL	
Classification	CONFIDENTIAL
Submitted to	Yash Pandya
Designation	Staff Security Engineer
Address	Hiteen District, 13516, Riyadh, Saudi Arabia
E-Mail	yash.pandya@leantech.me

Version Control			
Version	Date	Author	Description
0.1	Nov 12, 2024	Buguard Security Team	Initial Report



1.0	Nov 12, 2024	Fares Walid , Ziad Elsayed	Final Report Version
1.1	Nov 19, 2024	Fares Walid	Edited Report Version
1.2	Nov 20, 2024	Fares Walid	Edited Report Version



Contents

1.	Executive Summary.....	1
2.	Findings Summary.....	2
3.	Disclaimer.....	3
4.	Scope.....	4
4.1	Timeline.....	4
4.2	Targets.....	4
4.3	Limitations.....	5
4.4	Infrastructure Scanning.....	5
4.5	Used Tools.....	5
5.	Methodology.....	6
5.1	Information Gathering	7
5.2	Applications Spidering	7
5.3	Vulnerability Identifying.....	8
5.4	Penetration Testing.....	9
5.5	Final Reporting.....	9
6.	Coverage	10
6.1	OWASP Top 10 Coverage.....	10
6.1	API OWASP Top 10 Coverage.....	10
7.	Testing Coverage.....	11
8.	Findings	13
8.1	BAC Support user privilege can add test users - not an option over UI	13
8.2	BAC Lead Developer user privilege can access invite history	15
8.3	Signup inputs limitations bypass over UI	17
8.4	Business Logic Workflow: MFA Requirement Bypass.....	18
9.	Scoring System	22

1. Executive Summary

The purpose of the assessment is to determine security flaws for the **Lean Technologies KSA**.

The tests are carried out assuming the identity of an attacker or a user with malicious intent.
The overall threat level in the original test was found to be **Low**.

As a result of the penetration testing exercise, it was possible to confirm that.

A) The app in the test were **Vulnerable** to multiple security flaws, such as:

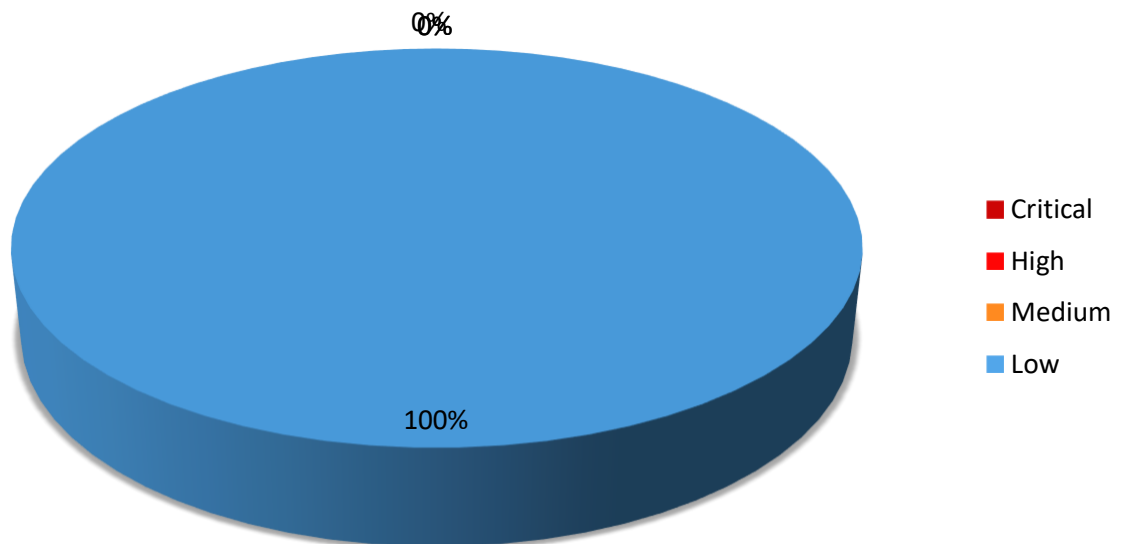
Severity	Title & Description	Status
Low	BAC Support user privilege can add test users - not an option over UI	Not Fixed.
	Unauthorized creation of test users can lead to potential abuse of the system, including bypassing usage limits or accessing restricted features.	
Low	BAC Lead Developer user privilege can access invite history	Not Fixed.
	Unauthorized access to invite history can lead to exposure of sensitive user information, such as email addresses and invitation details, which can be exploited for phishing or other malicious activities.	
Low	Signup inputs limitations bypass over UI	Not Fixed.
	Attackers can bypass input limitations to inject malicious data, potentially leading to security vulnerabilities such as SQL injection, XSS, or buffer overflow attacks.	
Low	Business Logic Workflow: 2fa requirement bypass	Not Fixed.
	This can be exploited by attackers to bypass the additional layer of security. This vulnerability undermines the security benefits provided by 2FA, making accounts susceptible to attacks even if users have 2FA enabled.	

Detailed vulnerabilities related to the applications are also included in this report corresponding to “**Critical**”, “**High**”, “**Medium**”, and “**Low**” level vulnerabilities require immediate actions.

2. Findings Summary

Value	Count
Critical	0
High	0
Medium	0
Low	4

Vulnerability Summary



3. Disclaimer

This security assessment was conducted on Lean Technologies asset with authentication on sandbox and production environment. The description of findings, recommendations, and risks was valid on the date of submission of this report. Any projection of the report's future is subject to risk due to the changes in the infrastructure architecture, and it may no longer reflect its logic and controls.

Buguard is not absolute and can never be guaranteed. New vulnerabilities are constantly being discovered, so monitoring, maintaining, and reviewing policy and practice as they relate to specific use cases and operating environments is necessary.

The information contained in this report is confidential and intended for Lean Technologies only. This report's unauthorized use, disclosure, copying, or alteration is strictly prohibited. Buguard will not be liable for direct, special, or indirect consequential damages arising from alteration of the contents of this report by a third party.



لين
LEAN



4. Scope

4.1 Timeline

Application	Start Date	End Date	Days of Engagement
Lean Technologies	Oct 24, 2024	Nov 03, 2024	7
API	Nov 04, 2024	Nov 10, 2024	6
IPs/Hosts	Nov 10, 2024	Nov 12, 2024	2

4.2 Targets

Description	URL
Lean Technologies	https://dev.sa.leantech.me
IPs/Hosts	dev.sa.leantech.me : 108.158.46.120 link.sa.leantech.me, app.sa.leantech.me, hyperlink-api.sa.leantech.me : 193.122.82.235 api.sa.leantech.me : 207.127.97.182 link.sandbox.sa.leantech.me, sandbox.sa.leantech.me, hyperlink-api.sandbox.sa.leantech.me, obksabank.sandbox.sa.leantech.me, auth.sandbox.sa.leantech.me : 193.122.65.127 connect.sa.leantech.me, auth.sa.leantech.me : 193.122.82.235 obksabank.leantech.me : 100.111.185.242 hyperlink-api.sa.leantech.me , api2.sa.leantech.me : 193.122.82.235
API	https://sandbox.sa.leantech.me https://api.sa.leantech.me https://api2.sa.leantech.me https://link.sa.leantech.me



4.3 Limitations

The results presented in this report represent the situation in a limited time frame. Any changes in the current architecture, code and/or configurations might lead to different results if the test is to be conducted again.

This penetration test does not include

- Distributed Denial of Service Attacks.
- Social Engineering.

4.4 Infrastructure Scanning

These scans were comprehensive and aimed at detecting any security vulnerabilities. The severity of the vulnerability assessment conducted were found to be **secure**.

The target's network's hosts and IP addresses were thoroughly scanned using:

- Nessus
- Nmap
- ssltest.sh
- Telnet
- Netcat

4.5 Used Tools

During the penetration testing of your project, a variety of specialized tools were employed to assess the security of your systems. Below is a brief overview of the key tools utilized:

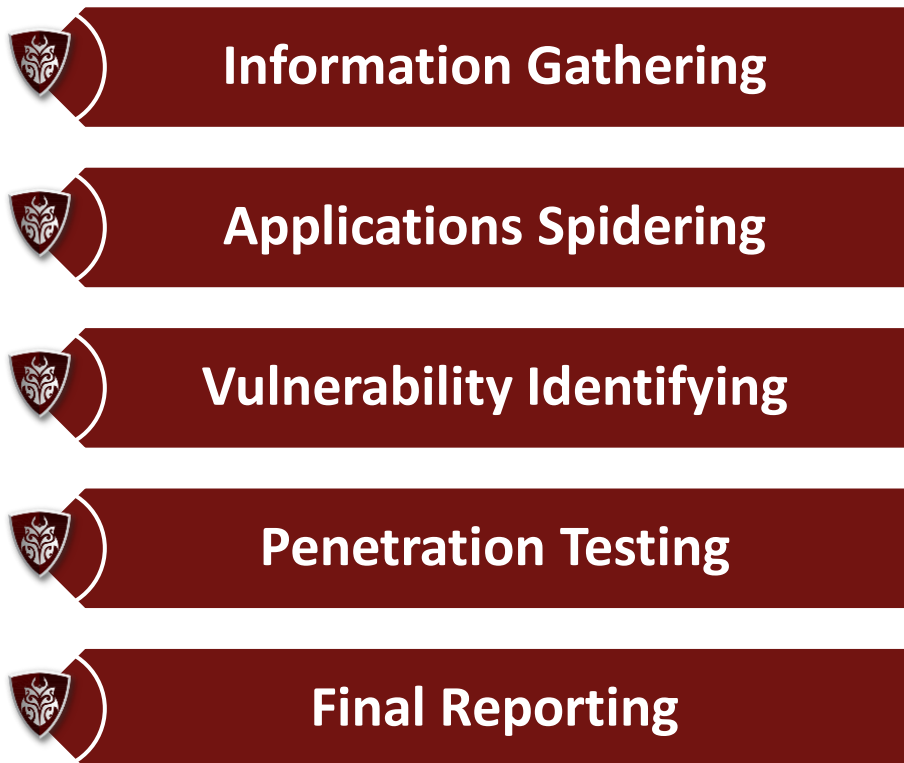
- Burp Suite
- API leaks
- Nuclei
- Ffuf
- Sqlmap
- Httpx
- httpprobe
- Waybackurls
- Katana
- gau

5. Methodology

Applications & APIs Penetration Test involves Buguard security researchers performing a thorough examination of the web-based applications for vulnerabilities in development and implementation including the 'OWASP Top 10' and 'SANS Top 25' in addition to the examination of the client applications.

Buguard follows a solid process based on an established well-known standard in the field of information security to deliver professional services to enterprises.

This task is composed of five phases:





5.1 Information Gathering

In the initial phase of our penetration testing process, we conduct extensive research on the target application to gain a deep understanding of its architecture, functionality, and supporting infrastructure. This includes reviewing any known vulnerabilities to identify potential areas of weakness in the application.

To fully understand the target's environment, we also create profiles of each server hosting the application. These profiles provide insight into the hardware and software configurations, network settings, and other relevant details of the servers. This allows us to assess the capabilities and limitations of the servers, as well as any vulnerabilities that may be present.

By thoroughly researching the target application and its environment, we can effectively plan and execute our testing efforts to identify and report on any vulnerabilities that may exist.

5.2 Applications Spidering

In this phase of the testing process, we conduct a comprehensive technical analysis of the target system in order to identify its vulnerabilities and potential points of exploitation.

This includes the following activities:

- Uncovering open ports and evaluating the services running on them
- Enumerating any websites and web services hosted on the server
- Inspecting the client application for vulnerabilities or weaknesses
- Examining the connections between the client and server for potential vulnerabilities
- Performing additional research and development to supplement our analysis
- Identifying high-priority issues and proposing effective solutions
- Gathering information about any remote services utilized by the system

Through this comprehensive analysis, we strive to fully understand the target system and prioritize our testing efforts to effectively identify and mitigate potential vulnerabilities.

5.3 Vulnerability Identifying

During the vulnerability identification phase, we perform a range of activities to detect exploitable weak points in the target system. These activities include:

- Testing authentication protocols and procedures
- Evaluating authorization controls and permissions
- Analyzing session management protocols and practices
- Verifying input validation processes and procedures
- Assessing error handling mechanisms and policies
- Evaluating the strength and effectiveness of cryptographic protocols and implementations
- Examining business logic for potential vulnerabilities or weaknesses
- Testing client-side components and processes

By thoroughly testing these various aspects of the target system, we aim to identify and prioritize vulnerabilities that may be exploited by attackers.

For the online services, the “**OWASP**” Top ten lists served as a guide and the domains tested for are listed below but not limited to them:

A1- Broken Access Control
A2- Cryptographic Failures
A3- Injection
A4- Insecure Design
A5- Security Misconfiguration
A6- Vulnerable and Outdated Components
A7- Identification and Authentication Failures
A8- Software and Data Integrity Failures
A9- Security Logging and Monitoring Failures
A10- Server-Side Request Forgery

OWASP Top 10 2021



5.4 Penetration Testing

After gathering information about the target network, we identify potential attack vectors and begin penetration testing activities. This phase includes the following activities:

Exploiting discovered vulnerabilities in the application and network using known methods such as:

- Malicious code injection
- Authentication bypass
- Authorization bypass
- Privilege escalation
- XML injection attacks
- XML external entity (XXE) attacks
- Race condition vulnerabilities
- Business logic vulnerabilities
- Attempting man-in-the-middle attack scenarios
- Attempting password cracking and/or pass-the-hash attacks
- Gathering information about remote services
- Testing the service with open-source web application auditing tools
- Conducting manual testing to discover in-depth vulnerabilities within services
- Verifying discovered vulnerabilities to minimize false positives and negatives
- Performing penetration testing according to approved documents

Through these activities, we aim to identify and exploit vulnerabilities in the target system in order to assess its security posture.

5.5 Final Reporting

Reporting the findings of the penetration tests is crucial to achieving the strategic goals and objectives of the testing process. Therefore, after completing the testing activities, we follow a documentation scheme to report our results to both technical and management audiences. This includes detailing the vulnerabilities that were discovered, the methods used to exploit them, and any recommendations for mitigating the identified risks. By thoroughly documenting and communicating our findings, we can ensure that the necessary steps are taken to address any vulnerabilities and improve the overall security posture of the target system.

6. Coverage

6.1 OWASP Top 10 Coverage

OWASP Category	Status
A1- Broken Access Control	FAIL
A2- Cryptographic Failures	PASS
A3- Injection	PASS
A4- Insecure Design	FAIL
A5- Security Misconfiguration	FAIL
A6- Vulnerable and Outdated Components	PASS
A7- Identification and Authentication Failures	FAIL
A8- Software and Data Integrity Failures	PASS
A9- Security Logging and Monitoring Failures	PASS
A10- Server-Side Request Forgery	PASS
OWASP Top 10 2021	

6.1 API OWASP Top 10 Coverage

OWASP Category	Status
A1- Broken Access Control	PASS
A2- Cryptographic Failures	PASS
A3- Injection	PASS
A4- Insecure Design	PASS
A5- Security Misconfiguration	PASS
A6- Vulnerable and Outdated Components	PASS
A7- Identification and Authentication Failures	PASS
A8- Software and Data Integrity Failures	PASS
A9- Security Logging and Monitoring Failures	PASS
A10- Server-Side Request Forgery	PASS
OWASP Top 10 2021	

7. Testing Coverage

Test Case	Status
Enumerate Applications on Webserver	PASS
Identify application entry points	PASS
Map execution paths through application	PASS
Fingerprint Web Application Framework	PASS
Fingerprint Web Application	PASS
Map Application Architecture	PASS
JWT Misconfigurations	PASS
Enumerate Infrastructure and Application Admin Interfaces	PASS
HTTP Methods	PASS
Information Disclosure	PASS
HTTP Strict Transport Security	PASS
User Registration Process	PASS
Account Enumeration and Guessable User Account	PASS
Weak or unenforced username policy	N/A
Permissions of Guest/Training Accounts	N/A
Account Suspension/Resumption Process	PASS
Default credentials	PASS
Weak lock out mechanism	PASS
Remember password functionality	PASS
Browser cache weakness	PASS
Weak password policy	PASS
Weak password change or reset functionalities	PASS
Directory traversal/file include	PASS
Privilege Escalation	FAIL
Insecure Direct Object References	PASS
Bypassing Session Management Schema	PASS
Cookies attributes	PASS
Session Fixation	PASS
Cross Site Request Forgery	PASS
Logout functionality	PASS
Session Timeout	PASS
Reflected Cross Site Scripting	PASS
Stored Cross Site Scripting	PASS
HTTP Verb Tampering	PASS
HTTP Parameter pollution	PASS
SQL Injection	PASS
LDAP Injection	N/A

XML Injection	PASS
SSI Injection	N/A
XPath Injection	N/A
Code Injection	PASS
Local File Inclusion	PASS
Remote File Inclusion	PASS
Command Injection	PASS
Buffer overflow	N/A
HTTP Splitting/Smuggling	PASS
Analysis of Error Codes	PASS
Business Logic Data Validation	FAIL
Ability to Forge Requests	PASS
Upload of Unexpected File Types	PASS
DOM based Cross Site Scripting	PASS
HTML Injection	PASS
Client-Side URL Redirect	PASS
CSS Injection	PASS
Client-Side Resource Manipulation	PASS
Cross Origin Resource Sharing	PASS
Clickjacking	PASS
Rate Limiting	PASS

8. Findings

8.1 BAC | Support user privilege can add test users - not an option over UI

Description

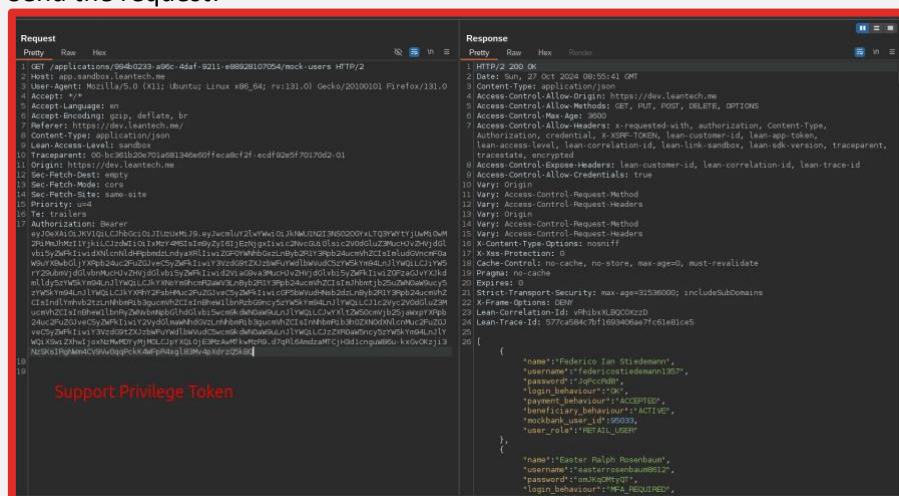
Broken Access Control (BAC) occurs when support user privileges allow adding test users, which is not an option available via the user interface (UI). During testing, we discovered that the system permits support users to add test users through direct API calls or other backend methods, bypassing UI restrictions. Unauthorized creation of test users can lead to potential abuse of the system, including bypassing usage limits or accessing restricted features.

Proof of Concept

1. As a Support Privilege user copy your Authorization token and your application UUID then replace it in the following request:

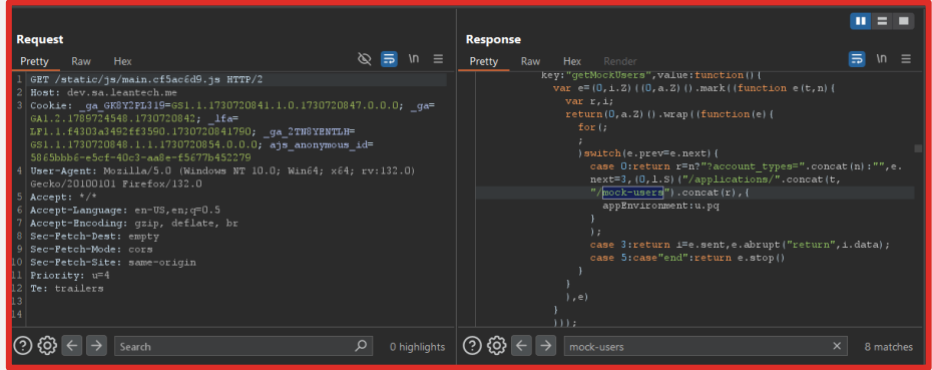
```
GET /applications/994b0233-a96c-4daf-9211-e88928107054/mock-users HTTP/2
Host: app.sandbox.leantech.me
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:131.0) Gecko/20100101 Firefox/131.0
Accept: */*
Accept-Language: en
Accept-Encoding: gzip, deflate, br
Referer: https://dev.leantech.me/
Content-Type: application/json
Lean-Access-Level: sandbox
Traceparent: 00-bc361b20e701a681346e60ffeca8cf2f-ecdf92e5f70170d2-01
Origin: https://dev.leantech.me
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-site
Priority: u=4
Te: trailers
Authorization: Bearer xxx
```

2. Send the request:



3. We can obtain these endpoint for the low privileges user from any JS file like:

<https://dev.sa.leantech.me/static/js/main.cf5ac6d9.js>

	
Impact	- This vulnerability undermines the integrity of the access control system, making it easier for attackers or malicious insiders to exploit other potential weaknesses. - It can lead to data pollution and inconsistencies, especially if test users are used for activities that should be restricted to specific roles or environments.
Recommendation	- Implement proper access control checks at the API and backend levels to ensure that only authorized actions are permitted, regardless of the UI. - Conduct a thorough review and audit of access control policies to ensure consistency between the UI and backend permissions.
Threat Level	Low
CVSS Score	CVSS:3.0/AV:N/AC:H/PR:L/UI:N/S:U/C:N/I:L/A:N/CR:X/IR:X/AR:X 3.1 (Low)
Affected URL	https://dev.sa.leantech.me /applications/{{app_id}}/mock-users
Status	Not fixed.

8.2 BAC | Lead Developer user privilege can access invite history

Description

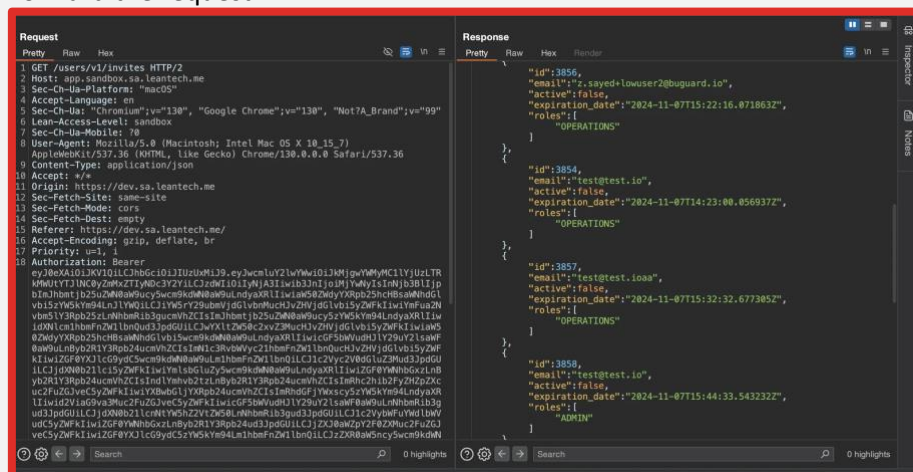
Broken Access Control (BAC) occurs when a Lead Developer user privilege allows access to invite history, which may not be intended or appropriate. During testing, we found that the system permits Lead Developer users to view the invite history, potentially exposing sensitive information such as email addresses, invitation statuses, and timestamps. Unauthorized access to invite history can lead to exposure of sensitive user information, such as email addresses and invitation details, which can be exploited for phishing or other malicious activities.

Proof of Concept

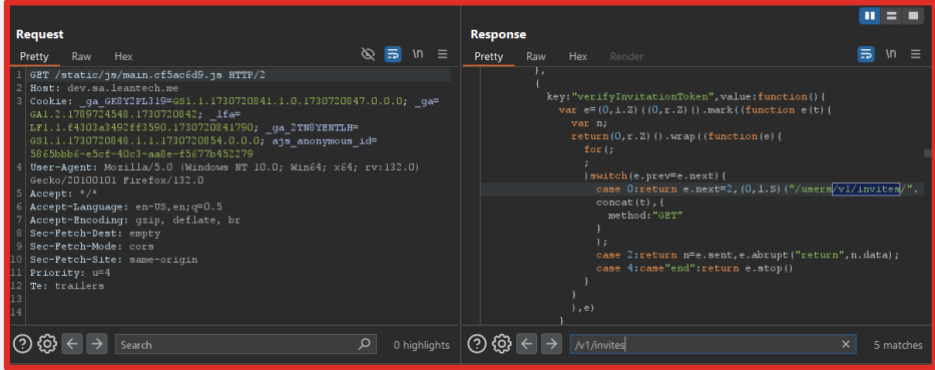
1. As a Lead Developer user copy your Authorization token and your application UUID then replace it in the following request:

```
GET /users/v1/invites HTTP/2
Host: app.sandbox.sa.leanotech.me
Sec-Ch-Ua-Platform: "macOS"
Accept-Language: en
Sec-Ch-Ua: "Chromium";v="130", "Google Chrome";v="130", "Not?A_Brand";v="99"
Lean-Access-Level: sandbox
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: https://dev.sa.leanotech.me
Sec-Fetch-Site: same-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://dev.sa.leanotech.me/
Accept-Encoding: gzip, deflate, br
Priority: u=1, i
Authorization: Bearer xxx
```

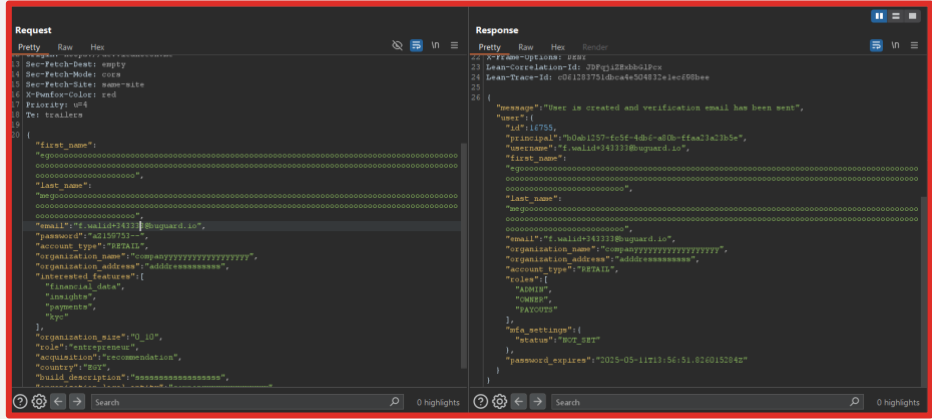
2. Forward the request



3. We can obtain these endpoints for the low privileges user from any JS file like: <https://dev.sa.leanotech.me/static/js/main.cf5ac6d9.js>

	
Impact	- This vulnerability can result in privacy violations, damaging user trust and potentially leading to legal and regulatory repercussions. - It undermines the overall security posture of the system, as it indicates potential weaknesses in the implementation and enforcement of access controls.
Recommendation	- Implement strict access control policies to ensure that only authorized roles can access the invite history. Review and adjust permissions to align with the principle of least privilege. - Use role-based access control (RBAC) to define and enforce access levels for different user roles, ensuring that sensitive information is only accessible to those who need it.
Threat Level	Low
CVSS Score	CVSS:3.0/AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N/CR:X/IR:X/AR:X 3.2 (Low)
Affected URL	https://dev.sa.leantech.me/users/v1/invites
Status	Not fixed.

8.3 Signup inputs limitations bypass over UI

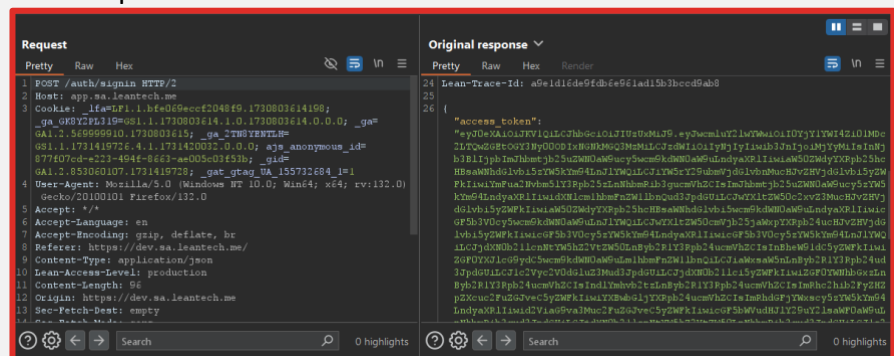
Description	Signup inputs limitations bypass over the UI is a vulnerability where an attacker can bypass input validation and restrictions enforced by the UI during the signup process. During testing, we discovered that the system fails to properly enforce input constraints at the server level, allowing attackers to submit invalid or malicious data by manipulating the client-side code. Attackers can bypass input limitations to inject malicious data, potentially leading to security vulnerabilities such as SQL injection, XSS, or buffer overflow attacks.
Proof of Concept	- it's not allowed in name to submit special chars and field of build description limited to 255 size only, bypassable from request - We have noticed too that this is not the only vuln parameter, but all fields like the first/last name are allowed to bypass limits. 
Impact	- Bypassing input constraints can disrupt the normal functioning of the application, leading to unexpected behavior, data corruption, or system crashes. - This vulnerability can result in unauthorized access, and compromise of sensitive information, damaging the service's integrity and user trust.
Recommendation	- Utilize input sanitization and escaping techniques to prevent injection attacks, ensuring that any user-supplied data is properly handled. - Conduct regular security testing and code reviews to identify and address potential input validation weaknesses and ensure comprehensive protection.
Threat Level	Low
CVSS Score	CVSS:3.0/AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:L/A:N/CR:X/IR:X/AR:X 3.1 (Low)
Affected URL	POST /auth/signup
Status	Not fixed.

8.4 Business Logic Workflow: MFA Requirement Bypass

Description

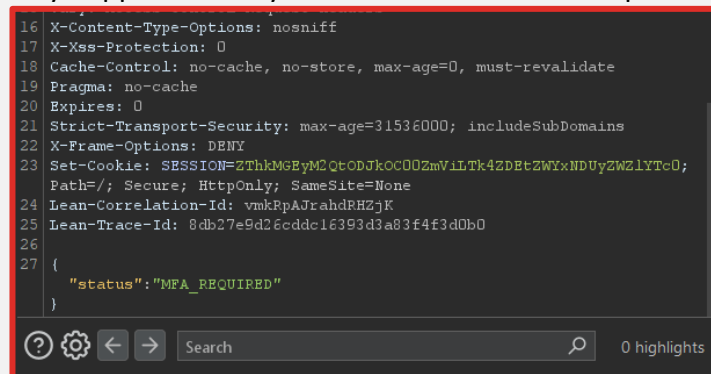
Two-factor authentication (2FA) requirement bypass is a vulnerability where an attacker can circumvent the 2FA mechanism, gaining unauthorized access to user accounts. During testing, we discovered that the system allows certain actions or access without enforcing the 2FA requirement, which can be exploited by attackers to bypass the additional layer of security. This vulnerability undermines the security benefits provided by MFA, making accounts susceptible to attacks even if users have 2FA enabled.

1. Sign-up for a new account, and confirm it via mail.
2. Login and intercept the request to copy your authorization token from response:



Proof of Concept

3. This in normal case, it reflects only a sessionId in cookies. (But this only happens when you have a confirmed MFA previously)



4. We can now Use this JWT and view any JS file and grep the endpoints from it and send them with your JWT directly.
5. The JS file: <https://dev.sa.leantech.me/static/js/main.cf5ac6d9.js>
6. We can search for: `/` to fetch all the endpoints with regex

9. All of this happened behind:

10. Or you can from a real account with MFA enabled just browse and send the requests with the unverified MFA JWT.

Impact

- This vulnerability can result in privacy violations and potentially leading to legal and regulatory repercussions.
- Bypassing the MFA forcing could be a business logic validation bypass, which puts the user's account in a high risk.



Recommendation	Enforce the MFA mechanism, on the login for the first time reflect a SESSIONID like the used me-ddddddddchanism in confirmed MFA accounts.
Threat Level	Low
CVSS Score	CVSS:3.0/AV:N/AC:H/PR:H/UI:N/S:U/C:L/I:L/A:N/CR:X/IR:X/AR:X 2.7 (Low)
Affected URL	POST /auth/signin /*
Status	Not fixed.

9. Scoring System

Common Vulnerability Scoring System (CVSS) Scoring System

After finding the vulnerabilities and confirming those in need of confirmation, it is necessary to use a scoring system that represents many aspects of the vulnerability to help the Customers make the right decisions.

We use the Common Vulnerability Scoring System (CVSS) Scoring System v3.0.

All details regarding the equations and calculations can be found on their official website at:

- <https://www.first.org/cvss/v3.0/specification-document>.

Qualitative Severity Rating Scale

For some purposes, it is useful to have a textual representation of the numeric Base, Temporal, And Environmental scores. All scores can be mapped to the qualitative ratings defined in the Below Table.

CVSS v3.0 Ratings*

Severity	Base Score Range
None	0.0
Low	0.1 – 3.9
Medium	4.0 – 6.9
High	7.0 – 8.9
Critical	9.0 – 10.0

*Vulnerability Severity Ratings provided by the National Vulnerability Database (nvd.nist.gov)